ROTEIRO DE AULA PRÁTICA

Introdução ao JavaScript Fundamentos e Interatividade na Web

Capacitação completa em manipulação de DOM, tratamento de eventos e lógica fundamental de programação para a Web.

Disciplina: Programação Web e Implantação de Modelos
Duração: 4 Horas | Pré-requisito: HTML & CSS

CONCEITOS E METAS

Objetivos da Aula: O que você vai dominar hoje

- **Entender o papel do JS**
  Compreender como o JavaScript adiciona comportamento e dinamismo a páginas estáticas.
- **Sintaxe e Lógica**
  Aprender a declarar variáveis, manipular tipos de dados, utilizar operadores e aplicar estruturas condicionais.
- **Criar Funções**
  Agrupar blocos lógicos reutilizáveis para organizar e isolar comportamentos na sua aplicação.
- **Domínio do DOM**
  Interagir e alterar a estrutura HTML e estilos CSS dinamicamente usando a API de objetos de documento (DOM).
- **Interação por Eventos**
  Capturar cliques de botões, entradas de texto em formulários e executar ações a partir dessas interações.
- **Projeto Prático Real**
  Construir uma Calculadora de Média interativa do zero, consolidando todo o conhecimento adquirido.

Entendendo o JavaScript: O Motor de Comportamento da Web

- **A Tríade Fundamental da Web**
  HTML define o que existe na página. CSS estabelece como ela aparece. JavaScript determina o que acontece.
- **O Conceito de Comportamento**
  O JavaScript é o que reage às ações do usuário. Ele permite que a página ganhe 'vida'.
- **Casos Práticos Comuns**
  Abrir e fechar menus flutuantes, validar formulários antes do envio, realizar cálculos na tela, atualizar partes da página sem recarregar e buscar dados de APIs externas.

HTML ↓ ESTRUTURA

O que existe na página?

CSS ↓ APARÊNCIA

Como aparece visualmente?

Escrevendo seu Primeiro Script e Usando o Console

- **Estrutura do Projeto**
  Crie uma pasta local contendo index.html, style.css e script.js para organizar os arquivos.

- **Conexão com script.js**
  O script JavaScript é importado através da tag <script src='script.js'></script> inserida antes de fechar a tag <body> do HTML.

- **A Ferramenta do Desenvolvedor**
  O console é essencial para depuração. Abra o navegador, vá em Inspecionar (F12) e acesse a aba 'Console' para ver a saída de console.log().

```
// Estrutura do Projeto
aula-javascript/
  ├── index.html
  ├── style.css
  └── script.js
```

```
// index.html
<!-- No arquivo index.html -->
<body>
    <h1>Minha primeira página com JS</h1>
    <p>Abra o console para ver o resultado.</p>

    <!-- Importante: conectar no final do body -->
    <script src="script.js"></script>
</body>
```

BLOCO 3 • VARIÁVEIS (25 MIN)

Armazenando Dados na Memória com let e const

• O que são Variáveis?

Espaços nomeados na memória do computador que servem para guardar e ler informações dinâmicas.

• Qual a diferença entre let e const?

Use let quando o valor precisar mudar (ex: idade que aumenta). Use const para valores e referências fixas que não devem ser alterados (ex: constante matemática ou elemento do DOM).

• Tipos de Dados Primitivos

JavaScript possui tipagem dinâmica. Tipos fundamentais incluem: String (textos), Number (números), Boolean (true/false) e Null (vazio voluntário).

```
// script.js
// Declarações com let (valores que alteram)
let idade = 25;
idade = 26; // Funciona normalmente!

// Declarações com const (valores imutáveis)
const faculdade = "FATEC";
faculdade = "Outra"; // Erro: reassignment!

// Principais Tipos Primitivos
let nome = "Ana";        // String
let preco = 12.99;       // Number (inteiro ou decimal)
let aprovado = true;     // Boolean
let telefone = null;     // Null (valor nulo/vazio)
```

Calculando e Comparando Dados com Operadores

- **Operadores Matemáticos**
  Realizam cálculos matemáticos clássicos: Soma (+), Subtração (-), Multiplicação (*) e Divisão (/).

- **Operadores de Comparação**
  Fazem testes lógicos que retornam valores booleanos (true ou false): maior (>), menor (<), diferente (!==) e igualdade.

- **O Segredo da Igualdade: == vs ===**
  O operador '===' compara o VALOR e o TIPO (estrito), enquanto '==' tenta converter os tipos antes de comparar. Como regra de segurança, prefira sempre ===.

```
// script.js

// Operações Aritméticas Básicas
let n1 = 10;
let n2 = 5;
console.log(n1 + n2); // 15
console.log(n1 * n2); // 50

// Comparações de Lógica
let idade = 20;
console.log(idade > 18);  // true
console.log(idade === 18); // false

// Comparação estrita vs flexível
console.log(5 == "5");   // true (apenas valor)
console.log(5 === "5");  // false (compara tipo)
console.log(5 !== "5");  // true (estrito)
```

Lógica e Decisão: Direcionando o Fluxo com if e else

- **Estruturas de Decisão**
  Instruções que alteram a execução do programa dependendo do resultado de uma condição (verdadeira ou falsa).

- **if, else e else if**
  if (se) inicia a verificação. else if permite testar regras intermediárias se a primeira for falsa. else trata todos os outros casos imprevistos.

- **Visual do Fluxo de Decisão**
  Condição lógica → É verdadeira? → Se SIM: executa ação X. Se NÃO: ignora ou executa ação alternativa Y.

```
// script.js
// Validação simples de idade
let idade = 20;
if (idade >= 18) {
    console.log("Maior de idade");
} else {
    console.log("Menor de idade");
}

// Condicionais encadeadas (várias regras)
let nota = 8;
if (nota >= 9) {
    console.log("Excelente!");
} else if (nota >= 6) {
    console.log("Aprovado");
} else {
    console.log("Reprovado");
}
```

Organizando Lógica em Blocos Reutilizáveis com Funções

- **O que são Funções?**
  Um bloco de código encapsulado com nome próprio, criado para executar uma tarefa específica sempre que convocado.
- **Parâmetros (Entradas)**
  Variáveis locais declaradas nos parênteses da função que servem para receber dados dinâmicos de fora.
- **O Comando return**
  Finaliza a execução da função e devolve o resultado final obtido para quem chamou a função de volta.

```
// script.js
// 1. Função Básica (Sem parâmetros)
function saudacao() {
    console.log("Olá!");
}
saudacao(); // Executa/chama a função

// 2. Função com Parâmetros
function saudarUsuario(nome) {
    console.log("Olá, " + nome + "!");
}
saudarUsuario("Maria"); // Saída: Olá, Maria!

// 3. Função com Retorno de valor
function somar(numero1, numero2) {
    let resultado = numero1 + numero2;
    return resultado; // Devolve o valor
}
let total = somar(10, 5); // total vira 15
```

A Ponte de Integração: Introdução ao DOM

- **O que significa DOM?**
  Document Object Model. É a representação que o navegador cria para o seu HTML, transformando a página em objetos que o JS consegue ler e manipular.

- **Acessando Elementos por ID**
  O método document.getElementById('id') permite capturar do HTML um elemento específico que tenha o atributo id correspondente.

- **Modificando Conteúdos Dinamicamente**
  Com a propriedade .textContent, é possível alterar de forma direta e visual o texto interno de qualquer elemento HTML em tempo de execução.

```
// index.html
<!-- No arquivo index.html -->
<h1 id="titulo">Olá mundo</h1>
```

```
// script.js
const titulo = document.getElementById("titulo");
console.log(titulo); // Exibe o objeto no console

// Modifica o conteúdo na tela instantaneamente!
titulo.textContent = "JavaScript funcionando!";
```

★ A mágica acontece aqui: A página web estática agora muda visualmente por conta de uma linha de script!

Selecionando Qualquer Elemento com querySelector

- **O Poder do querySelector**
  É a forma moderna e mais recomendada de buscar elementos HTML. Ele é extremamente flexível porque utiliza os mesmos seletores do CSS.

- **Buscando por IDs (#)**
  Utilize a mesma sintaxe de ID do CSS. Exemplo: document.querySelector('#titulo') busca o elemento com id='titulo'.

- **Buscando por Classes (.)**
  Utilize a mesma sintaxe de classes do CSS. Exemplo: document.querySelector('.texto') busca o primeiro elemento com class='texto'.

```
// index.html
<!-- Estrutura no index.html -->
<h1 id="titulo">Minha página</h1>
<p class="texto">
    Aprendendo JavaScript
</p>
```

```
// script.js
// Seleção direta usando padrões do CSS
const titulo = document.querySelector("#titulo");
const texto = document.querySelector(".texto");

// Pronto! Agora você pode manipular ambos:
titulo.textContent = "Novo Título";
texto.textContent = "Alteramos o parágrafo!";
```

Ação e Reação: Capturando Cliques com Eventos

- **O que são Eventos?**
  Qualquer sinalização que ocorre no navegador gerada pelo usuário ou sistema (cliques, digitação, rolagem, etc.).
- **A Escuta: addEventListener()**
  É o método que vinculamos a um elemento para 'escutar' quando um evento específico acontece e disparar uma resposta.
- **A Função de Retorno (Callback)**
  A função passada como argumento dentro do escutador de eventos roda exatamente no instante do acontecido.

```
// index.html
<!-- HTML de gatilho -->
<button id="botao">Clique aqui</button>
```

```
// script.js
// Seleciona o botão no DOM
const botao = document.querySelector("#botao");
const titulo = document.querySelector("#titulo");

// Vincula o escutador de clique
botao.addEventListener("click", function() {
    console.log("O botão foi clicado!");
    titulo.textContent = "Você clicou no botão!";
});
```

Estilo em Movimento: Alterando CSS Dinamicamente

- **A Propriedade style**
  JavaScript permite acessar as definições CSS inline de um elemento através do atributo '.style' do objeto selecionado.
- **Regra de Escrita camelCase**
  No CSS escrevemos 'font-size' (com traço). No JavaScript, como o traço significa subtração, convertemos para camelCase: 'fontSize'.
- **O Papel de Cada Tecnologia**
  HTML: Cria e estrutura o elemento. CSS: Define as propriedades de layout e aparência de início. JS: Altera cores, tamanhos e exibição em tempo de execução.

```
// index.html
<h1 id="titulo">Minha página</h1>
<button id="botao">Mudar aparência</button>
```

```
// script.js
const titulo = document.querySelector("#titulo");
const botao = document.querySelector("#botao");

botao.addEventListener("click", function() {
    // Altera propriedades de estilo em tempo real!
    titulo.style.color = "blue";
    titulo.style.fontSize = "50px";
});
```

Capturando Entradas do Usuário e Validando Dados

- **A Propriedade .value**
  Permite capturar o texto exato que o usuário digitou em um campo de entrada (input) no formulário do HTML.
- **Construindo Fluxos Completos**
  A aplicação agora atua de ponta a ponta: recebe uma entrada, executa processamentos e devolve saídas personalizadas.
- **Garantindo Qualidade com Validação**
  Conecta as estruturas condicionais (if/else) aprendidas para verificar se o campo está em branco antes de avançar.

```
// index.html
<input type="text" id="nome" placeholder="Digite seu nome">
<button id="botao">Enviar</button>
<p id="resultado"></p>
```

```
// script.js
const campoNome = document.querySelector("#nome");
const botao = document.querySelector("#botao");
const resultado = document.querySelector("#resultado");

botao.addEventListener("click", function() {
    const nome = campoNome.value;
    if (nome === "") {
        resultado.textContent = "Digite seu nome.";
    } else {
        resultado.textContent = "Olá, " + nome + "!";
    }
});
```

Projeto Prático Guiado: Calculadora de Média

- **O Desafio Prático**
  Reunir toda a estrutura HTML, estilização visual CSS e controle comportamental JS em uma aplicação de cálculo real.

- **A Importância do tipo Number()**
  Por padrão, o valor capturado de um input de número ainda é tratado como texto. Devemos convertê-lo usando a função nativa Number() antes de somar, caso contrário o JS apenas concatenará os valores.

- **Melhorias Visuais e de Saída**
  O resultado deve exibir a média com uma casa decimal formatada usando .toFixed(1) e dizer a situação lógica.

```
// script.js
const botao = document.querySelector("#calcular");
botao.addEventListener("click", function() {
  const nome = document.querySelector("#nome").value;
  // Conversão explícita para Number é obrigatória!
  const nota1 = Number(document.querySelector("#nota1").value);
  const nota2 = Number(document.querySelector("#nota2").value);
  const nota3 = Number(document.querySelector("#nota3").value);
  const resultado = document.querySelector("#resultado");

  const media = (nota1 + nota2 + nota3) / 3;

  if (media >= 6) {
    resultado.textContent = nome + " - Média: " +
      media.toFixed(1) + " - APROVADO";
  } else {
    resultado.textContent = nome + " - Média: " +
      media.toFixed(1) + " - REPROVADO";
  }
});
```